



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE-05 Creación, Lectura y Escritura de Ficheros
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumno participante:	Chacha Chango Victor Manuel
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Manipular archivos mediante métodos y clases que proporciona el lenguaje de programación JAVA

Específicos:

- Implementar las clases de la librería java.io (como FileWriter y PrintWriter) para la creación y escritura de archivos de texto, garantizando la persistencia de los datos ingresados por el usuario.
- Desarrollar algoritmos de lectura secuencial utilizando clases como BufferedReader o Scanner, para extraer, separar (mediante el método split) y procesar correctamente la información almacenada en los ficheros .txt.
- Aplicar los principios de la Programación Orientada a Objetos (POO) encapsulando la lógica de manipulación de archivos dentro de métodos específicos de cada clase, asegurando un código limpio, modular y reutilizable.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas
- se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos



Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- - Computador o laptop.
- - Plataforma educativa de la UTA.
- Internet para consultas.
- - Material de escritorio.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
 - ☐ Simuladores y laboratorios virtuales
 - ☐ Aplicaciones educativas
 - ☒ Recursos audiovisuales
 - ☐ Gamificación
 - ☐ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

2.7 Resultados obtenidos

EJERCICIO 1: REGISTRO DE ESTUDIANTES

Enunciado:

Crear un programa que permita registrar estudiantes en un archivo de texto.
Requerimientos: Solicitar cédula, nombre, carrera y promedio. Guardar en estudiantes.txt.
Aplicar POO, métodos, y archivos.

Análisis del problema:

Se debe construir la clase Estudiante con atributos privados. Mediante un menú o entrada de datos, el usuario ingresará la información. Luego, se utilizarán métodos de escritura de archivos (ofstream en C++ y FileWriter en Java) utilizando el modo 'append' (añadir) para no sobrescribir los registros previos, asegurando la persistencia.

Variables de entrada, salida y proceso:

Entrada: cedula (Cadena), nombre (Cadena), carrera (Cadena), promedio (Real).
Proceso: Instanciación de objeto, apertura de flujo de escritura en modo añadir (append).
Salida: Confirmación en pantalla y escritura en estudiantes.txt.

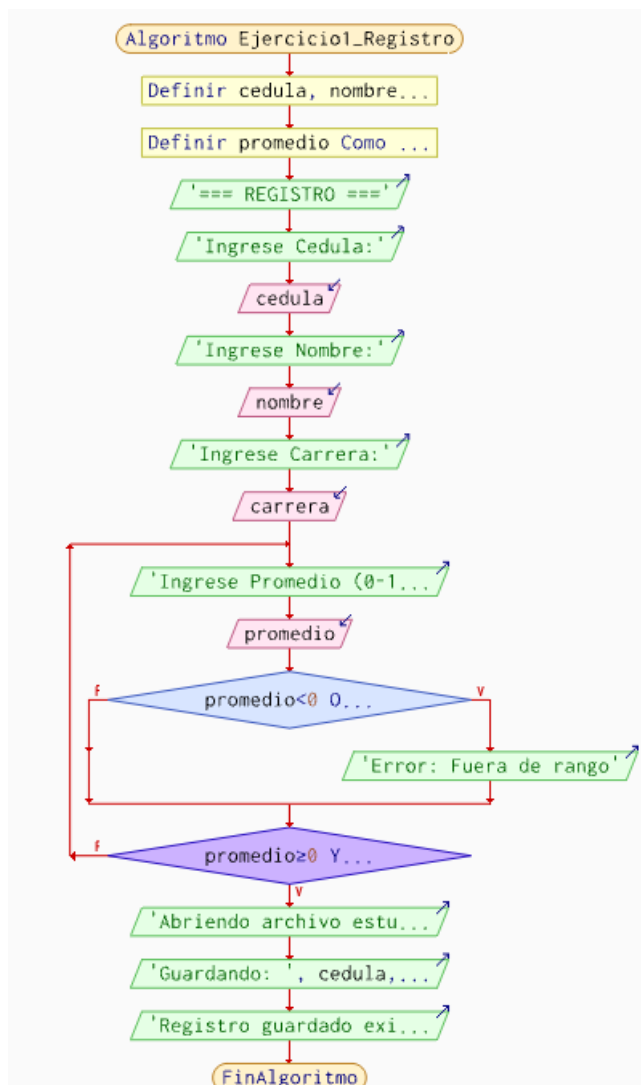
Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio1_Registro
Definir cedula, nombre, carrera Como Cadena
Definir promedio Como Real
Escribir "=== REGISTRO ==="
Escribir "Ingrese Cedula:"
```



Leer cedula
Escribir "Ingrese Nombre:"
Leer nombre
Escribir "Ingrese Carrera:"
Leer carrera
Repetir
 Escribir "Ingrese Promedio (0-10):"
 Leer promedio
 Si promedio < 0 O promedio > 10 Entonces
 Escribir "Error: Fuera de rango"
 FinSi
Hasta Que promedio >= 0 Y promedio <= 10
Escribir "Abriendo archivo estudiantes.txt en modo Append..."
Escribir "Guardando: ", cedula, ",", nombre, ",", carrera, ",", promedio
Escribir "Registro guardado exitosamente."
FinAlgoritmo

Diagrama de Flujo





Código en C++:

```
#include <iostream>
#include <fstream>
using namespace std;
class Estudiante {
private:
    string cedula, nombre, carrera; double promedio;
public:
    Estudiante(string c, string n, string car, double p) {
        cedula=c; nombre=n; carrera=car; promedio=p;
    }
    void guardar() {
        ofstream arch("estudiantes.txt", ios::app);
        if(arch.is_open()){
            arch<<cedula<<","<<nombre<<","<<carrera<<","<<promedio<<"\n";
            arch.close();
            cout<<"Estudiante guardado.\n";
        }
    }
};

int main() {
    string c, n, car; double p;
    cout<<"Cedula: "; cin>>c; cout<<"Nombre: "; cin>>n;
    cout<<"Carrera: "; cin>>car; cout<<"Promedio: "; cin>>p;
    Estudiante est(c, n, car, p);
    est.guardar();
    return 0;
}
```

Código en Java:

```
import java.io.*;
import java.util.Scanner;
class Estudiante {
    private String cedula, nombre, carrera; private double promedio;
    public Estudiante(String c, String n, String car, double p) {
        cedula=c; nombre=n; carrera=car; promedio=p;
    }
    public void guardar() {
        try(FileWriter fw = new FileWriter("estudiantes.txt", true);
            PrintWriter pw = new PrintWriter(fw)) {
            pw.println(cedula+","+nombre+","+carrera+","+promedio);
            System.out.println("Estudiante guardado.");
        }
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



```
        } catch(IOException e) { System.out.println("Error"); }  
    }  
}  
  
public class Main1 {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Cedula: "); String c = sc.next();  
        System.out.print("Nombre: "); String n = sc.next();  
        System.out.print("Carrera: "); String car = sc.next();  
        System.out.print("Promedio: "); double p = sc.nextDouble();  
        Estudiante est = new Estudiante(c, n, car, p);  
        est.guardar();  
    }  
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Ingreso exitoso	Ced: 1850024298, Nom: Victor, Car: Software, Prom: 9.5	Apertura en modo append, concatena cadena con comas.	Archivo estudiantes.txt actualizado. Mensaje: 'Guardado'
2. Nuevo ingreso	Ced: 1802807030, Nom: Segundo, Car: Software, Prom: 8.5	El modo append evita el borrado de la linea de Victor.	Mensaje: 'Guardado'. 2 líneas en txt.
3. Falla apertura	Sin permisos en el directorio	try-catch captura excepcion / if(arch.is_open) da falso	Mensaje de 'Error al guardar'



EJERCICIO 2: LECTURA DE ARCHIVO DE ESTUDIANTES

Enunciado:

Leer el archivo generado en el ejercicio 1. Mostrar todos los registros, contar cuántos existen y calcular el promedio general.

Análisis del problema:

La clase principal debe abrir el archivo 'estudiantes.txt' en modo lectura (ifstream en C++ o BufferedReader en Java). Mediante un ciclo 'mientras no sea fin de archivo', se extraerán los datos separándolos por la coma. Se usarán acumuladores para sumar los promedios y contadores para calcular el promedio total al finalizar.

Variables de entrada, salida y proceso:

Entrada: Datos leídos del archivo de texto.

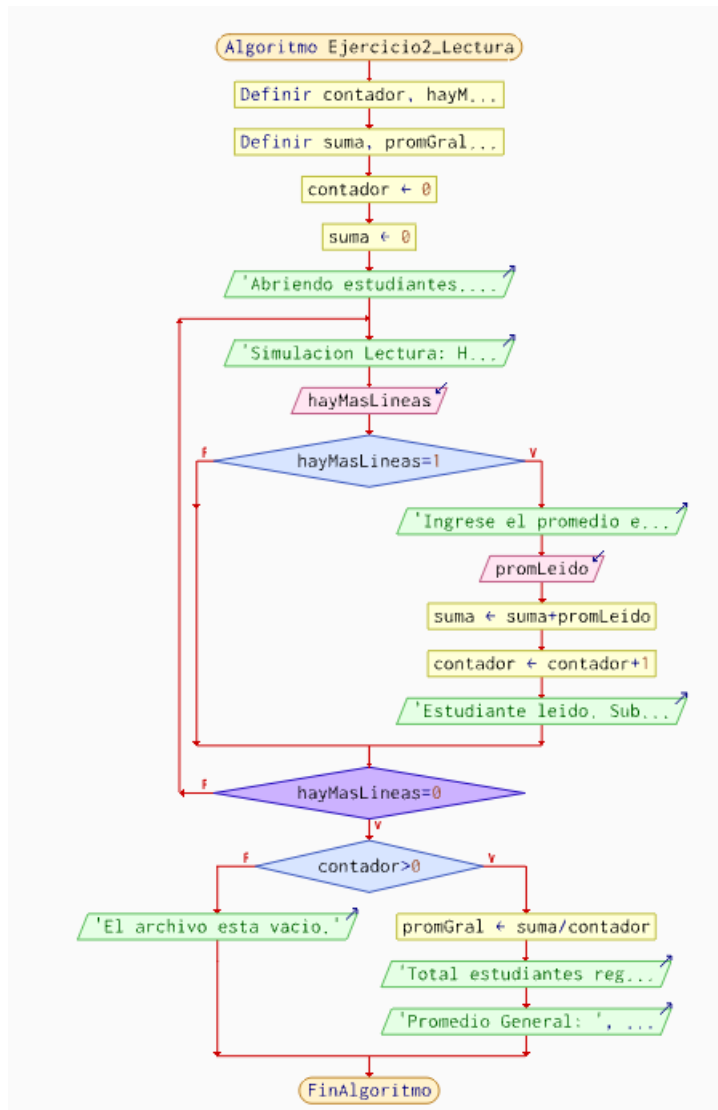
Proceso: Ciclo While, contadorEstudiantes (Entero), sumaPromedios (Real),
 $\text{promedioGral} = \text{sumaPromedios} / \text{contadorEstudiantes}$.

Salida: Lista en consola, conteo total, promedio general.

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio2_Lectura
  Definir contador, hayMasLineas Como Entero
  Definir suma, promGral, promLeido Como Real
  contador <- 0
  suma <- 0
  Escribir "Abriendo estudiantes.txt en modo Lectura..."
  Repetir
    Escribir "Simulacion Lectura: Hay otra linea en el archivo? (1=Si, 0=No)"
    Leer hayMasLineas
    Si hayMasLineas = 1 Entonces
      Escribir "Ingresa el promedio extraido de esta linea:"
      Leer promLeido
      suma <- suma + promLeido
      contador <- contador + 1
      Escribir "Estudiante leido. Subtotal acumulado: ", suma
    FinSi
  Hasta Que hayMasLineas = 0
  Si contador > 0 Entonces
    promGral <- suma / contador
    Escribir "Total estudiantes registrados: ", contador
    Escribir "Promedio General: ", promGral
  Sino
    Escribir "El archivo esta vacio."
  FinSi
FinAlgoritmo
```

Diagrama de Flujo



Código en C++:

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
using namespace std;
void leerArchivo() {
    ifstream arch("estudiantes.txt");
    string linea, c, n, car, p_str;
    int cont = 0; double suma = 0;
    while(getline(arch, linea)) {
        stringstream ss(linea);
        getline(ss, c, ','); getline(ss, n, ',');
        getline(ss, car, ','); getline(ss, p_str, ',');
        double p = stod(p_str);
```



```
cout<<"Ced: "<<c<<" Nom: "<<n<<" Prom: "<<p<<"\n";
suma += p; cont++;
}
arch.close();
if(cont>0) {
    cout<<"Total: "<<cont<<" | Promedio Gral: "<<(suma/cont)<<"\n";
}
}
int main() { leerArchivo(); return 0; }
```

Código en Java:

```
import java.io.*;
public class Main2 {
    public static void main(String[] args) {
        try(BufferedReader br = new BufferedReader(new
        FileReader("estudiantes.txt"))) {
            String linea; int cont = 0; double suma = 0;
            while((linea = br.readLine()) != null) {
                String[] d = linea.split(",");
                double p = Double.parseDouble(d[3]);
                System.out.println("Ced: "+d[0]+" Nom: "+d[1]+" Prom: "+p);
                suma += p; cont++;
            }
            if(cont>0) {
                System.out.println("Total: "+cont+" | Promedio Gral: "+(suma/cont));
            }
        } catch(IOException e) { System.out.println("No se pudo leer."); }
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Archivo con datos	estudiantes.txt tiene 2 filas (promedios 9.5 y 8.5)	Suma = 18.0, Cont = 2, PromGral = 9.0	Muestra los 2 alumnos. Total: 2, Promedio Gral: 9.0
2. Archivo vacio	estudiantes.txt existe pero sin lineas	Ciclo while no se ejecuta, cont = 0	No imprime listado. Evita división por cero.
3. Archivo inexistente	estudiantes.txt borrado	Falla el FileReader / ifstream	Muestra mensaje de error 'No se pudo leer'



EJERCICIO 3: SISTEMA DE INVENTARIO BÁSICO

Enunciado:

Crear un sistema para registrar productos. Guardar código, nombre, cantidad y precio en inventario.txt. Calcular el valor total del inventario.

Análisis del problema:

Se requiere una clase Producto. Un método debe permitir guardar la data en modo append. Otro método estático leerá el archivo de texto secuencialmente, multiplicará la cantidad por el precio de cada fila para obtener el subtotal de ese producto, y lo sumará a un total general del inventario, cumpliendo el cálculo solicitado.

Variables de entrada, salida y proceso:

Entrada: codigo, nombre, cantidad, precio.

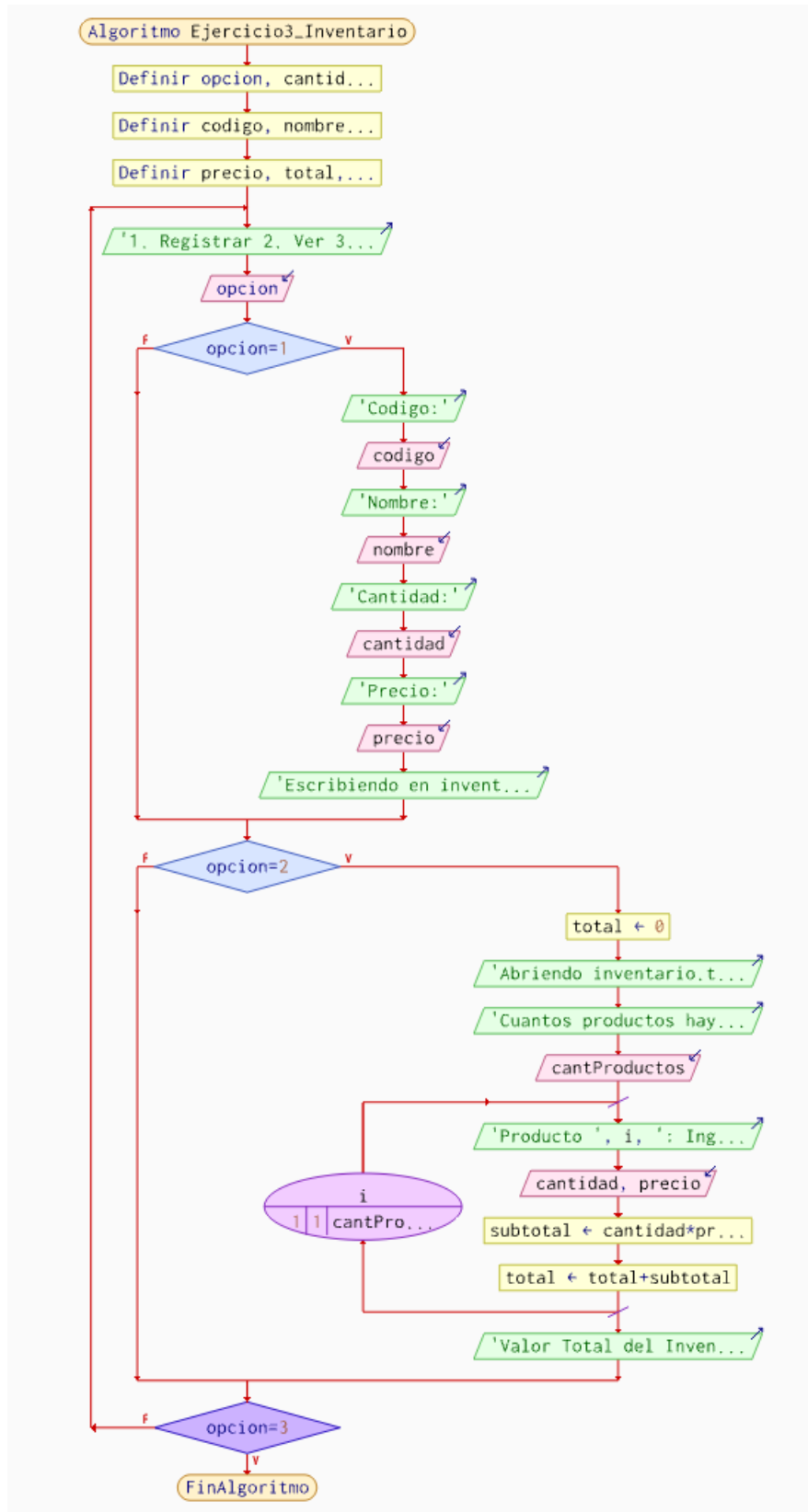
Proceso: $\text{subtotal} = \text{cantidad} * \text{precio}$, $\text{totalInventario} = \text{totalInventario} + \text{subtotal}$.

Salida: Producto guardado, lista de inventario y total global (\$).

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio3_Inventario
  Definir opcion, cantidad, cantProductos, i Como Entero
  Definir codigo, nombre Como Cadena
  Definir precio, total, subtotal Como Real
  Repetir
    Escribir "1. Registrar 2. Ver 3. Salir"
    Leer opcion
    Si opcion = 1 Entonces
      Escribir "Codigo:"; Leer codigo
      Escribir "Nombre:"; Leer nombre
      Escribir "Cantidad:"; Leer cantidad
      Escribir "Precio:"; Leer precio
      Escribir "Escribiendo en inventario.txt..."
    FinSi
    Si opcion = 2 Entonces
      total <- 0
      Escribir "Abriendo inventario.txt..."
      Escribir "Cuantos productos hay en el archivo?"
      Leer cantProductos
      Para i <- 1 Hasta cantProductos Con Paso 1 Hacer
        Escribir "Producto ", i, ": Ingrese cantidad y precio leidos:"
        Leer cantidad, precio
        subtotal <- cantidad * precio
        total <- total + subtotal
      FinPara
      Escribir "Valor Total del Inventario: $", total
    FinSi
  Hasta Que opcion = 3
FinAlgoritmo
```

Diagrama de Flujo



Código en C++:



```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
using namespace std;
class Producto {
    string cod, nom; int cant; double precio;
public:
    Producto(string c, string n, int ca, double p):cod(c),nom(n),cant(ca),precio(p){}
    void guardar(){
        ofstream arch("inventario.txt", ios::app);
        arch<<cod<<" "<<nom<<" "<<cant<<" "<<precio<<"\n";
        arch.close();
    }
    static void mostrar(){
        ifstream arch("inventario.txt");
        string linea, c, n, ca_s, p_s; double total=0;
        while(getline(arch, linea)){
            stringstream ss(linea);
            getline(ss,c,','); getline(ss,n,',');
            getline(ss,ca_s,','); getline(ss,p_s,',');
            int cant = stoi(ca_s); double pre = stod(p_s);
            double sub = cant * pre; total += sub;
            cout<<c<<" "<<n<<" Subtotal: $"<<sub<<"\n";
        }
        cout<<"Total Inventario: $"<<total<<"\n";
    }
};
int main() {
    Producto p("A1","Mouse",10,15.5); p.guardar();
    Producto::mostrar(); return 0;
}
```

Código en Java:

```
import java.io.*;
class Producto {
    String cod, nom; int cant; double precio;
    public Producto(String c, String n, int ca, double p) {
        cod=c; nom=n; cant=ca; precio=p;
    }
    public void guardar() {
        try(PrintWriter pw = new PrintWriter(new FileWriter("inventario.txt",true))){
            pw.println(cod+","+nom+","+cant+","+precio);
        }catch(Exception e){}
    }
}
```



```
}  
public static void mostrar() {  
    try(BufferedReader br = new BufferedReader(new  
        FileReader("inventario.txt"))){  
        String l; double t=0;  
        while((l=br.readLine())!=null){  
            String[] d = l.split(",");  
            int c = Integer.parseInt(d[2]); double p = Double.parseDouble(d[3]);  
            double sub = c * p; t += sub;  
            System.out.println(d[0]+" "+d[1]+" Sub: $" +sub);  
        }  
        System.out.println("Total Inv: $" +t);  
    } catch(Exception e){}  
}  
}  
public class Main {  
    public static void main(String[] args){  
        Producto p = new Producto("A1","Mouse",10,15.5); p.guardar();  
        Producto.mostrar();  
    }  
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Registrar prod	A1, Mouse, 10, 15.5	Append en txt	Confirmación de guardado
2. Cálculo total	Lee A1, 10, 15.5	$10 * 15.5 = 155$	Sub: \$155. Total Inv: \$155
3. Varios prods	Agrega A2, Teclado, 2, 20.0	$155 + (2*20) = 195$	Total Inv: \$195



EJERCICIO 4: AGENDA TELEFÓNICA

Enunciado:

Crear una agenda de contactos. Guardar nombre, teléfono y correo. Agregar, buscar contactos y mostrar todos.

Análisis del problema:

Implementar la clase Contacto. El requerimiento clave es la Búsqueda. En lugar de solo iterar e imprimir, el algoritmo de lectura debe comparar la subcadena de búsqueda ingresada por el usuario con cada línea leída del archivo (usando contains() en Java o find() en C++). Si coincide, se imprime; si recorre todo y no hay coincidencia, notifica.

Variables de entrada, salida y proceso:

Entrada: nombre, telefono, correo, nombreBuscar.

Proceso: Ciclo de lectura con condicional de coincidencia de subcadenas, uso de bandera booleana.

Salida: Contacto(s) encontrado(s) o mensaje de 'No existe'.

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio4_Agenda
  Definir opcion, lineasTotales, i Como Entero
  Definir nombre, telefono, correo, busqueda, lineaLeida Como Cadena
  Definir encontrado Como Logico
  Repetir
    Escribir "1. Agregar 2. Buscar 3. Salir"
    Leer opcion
    Si opcion = 1 Entonces
      Escribir "Nombre:"; Leer nombre
      Escribir "Telefono:"; Leer telefono
      Escribir "Correo:"; Leer correo
      Escribir "Escribiendo en agenda.txt..."
    FinSi
    Si opcion = 2 Entonces
      Escribir "Ingresa nombre a buscar:"
      Leer busqueda
      encontrado <- Falso
      Escribir "Cuantas lineas tiene el archivo?"
      Leer lineasTotales
      Para i <- 1 Hasta lineasTotales Con Paso 1 Hacer
        Escribir "Ingresa el nombre de la linea ", i, ":"
        Leer lineaLeida
        Si lineaLeida = busqueda Entonces
          Escribir "Encontrado: ", lineaLeida
          encontrado <- Verdadero
        FinSi
      FinPara
    Si encontrado = Falso Entonces
```

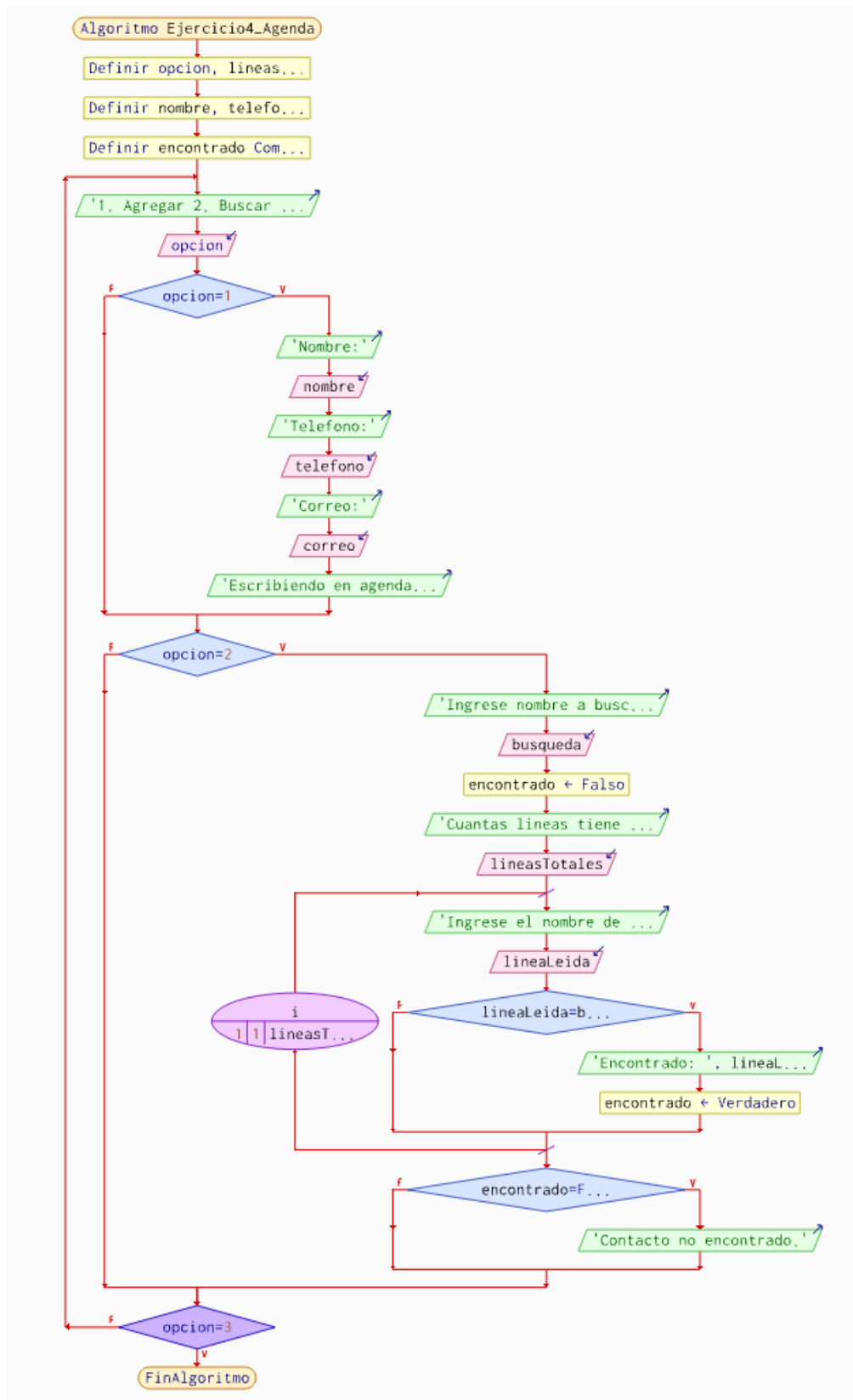


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



Escribir "Contacto no encontrado."
FinSi
FinSi
Hasta Que opcion = 3
FinAlgoritmo

Diagrama de Flujo



Código en C++:

```
#include <iostream>
#include <fstream>
#include <string>
```



```
using namespace std;
void agregar(string n, string t, string c) {
    ofstream arch("agenda.txt", ios::app);
    arch<<n<<" "<<t<<" "<<c<<"\n"; arch.close();
}
void buscar(string nombreBuscado) {
    ifstream arch("agenda.txt");
    string linea; bool enc = false;
    while(getline(arch, linea)) {
        if(linea.find(nombreBuscado) != string::npos) {
            cout<<"Encontrado: "<<linea<<"\n"; enc = true;
        }
    }
    if(!enc) cout<<"No existe el contacto.\n";
}
int main() {
    agregar("Victor Chacha", "09999999", "v@uta.edu.ec");
    buscar("Victor");
    return 0;
}
```

Código en Java:

```
import java.io.*;
public class Agenda {
    public static void agregar(String n, String t, String c) {
        try(PrintWriter pw = new PrintWriter(new FileWriter("agenda.txt",true))) {
            pw.println(n+" "+t+" "+c);
        } catch(Exception e){}
    }
    public static void buscar(String nom) {
        try(BufferedReader br = new BufferedReader(new
        FileReader("agenda.txt"))){
            String linea; boolean enc = false;
            while((linea = br.readLine())!=null){
                if(linea.toLowerCase().contains(nom.toLowerCase())) {
                    System.out.println("Encontrado: "+linea); enc = true;
                }
            }
            if(!enc) System.out.println("No existe.");
        } catch(Exception e){}
    }
    public static void main(String[] args){
        agregar("Victor Chacha", "09999999", "v@uta.edu.ec");
        buscar("Victor");
    }
}
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



}

}

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Agregar	Victor, 099, v@uta	Escribe linea en txt	Agenda actualizada
2. Buscar acierto	Buscar: 'Victor'	contains('Victor') da true	Imprime los datos de Victor
3. Buscar fallo	Buscar: 'Pedro'	lee todo, contains da false	Imprime 'No existe'



EJERCICIO 5: SISTEMA DE NOTAS

Enunciado:

Registrar calificaciones de estudiantes. Guardar nombre y 3 notas. Calcular promedio y determinar si aprueba (≥ 7) o reprueba.

Análisis del problema:

Similar al ejercicio 1, pero la lógica de negocio (matemática y condicional) se delega a la clase antes de guardar en el archivo. El método evaluador calculará el promedio, y si es ≥ 7 asume estado 'Aprobado', sino 'Reprobado'. Estos datos consolidados se escriben en notas.txt.

Variables de entrada, salida y proceso:

Entrada: nombre, n1, n2, n3.

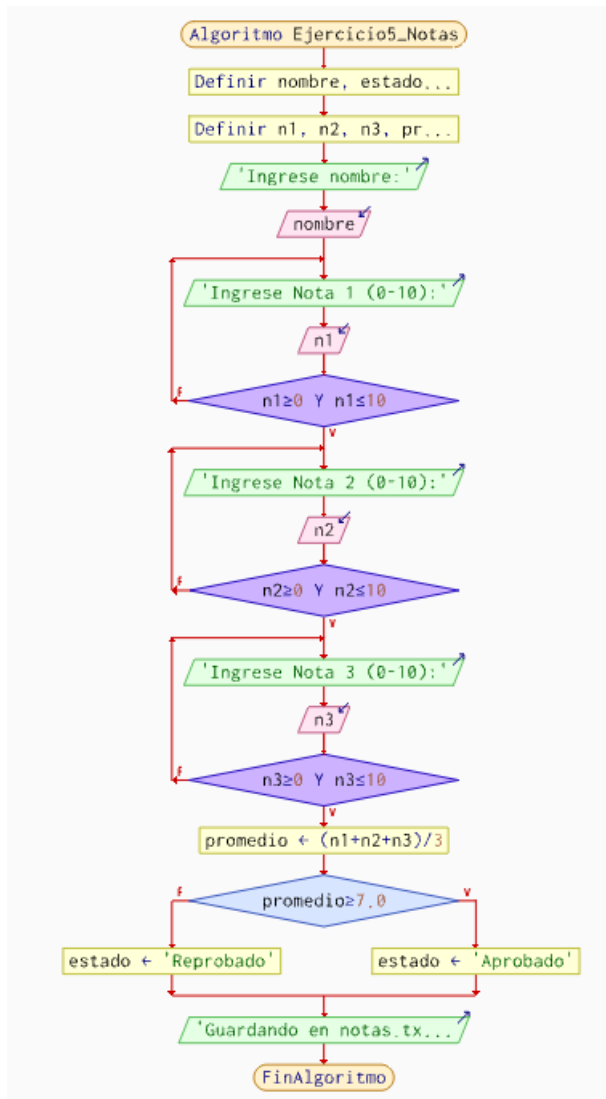
Proceso: $\text{prom} = (n1 + n2 + n3) / 3$. Si $\text{prom} \geq 7$ estado='Aprobado'. Escribir en archivo.

Salida: Archivo texto con el reporte consolidado.

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio5_Notas
  Definir nombre, estado Como Cadena
  Definir n1, n2, n3, promedio Como Real
  Escribir "Ingrese nombre:"
  Leer nombre
  Repetir
    Escribir "Ingrese Nota 1 (0-10):"
    Leer n1
  Hasta Que n1 >= 0 Y n1 <= 10
  Repetir
    Escribir "Ingrese Nota 2 (0-10):"
    Leer n2
  Hasta Que n2 >= 0 Y n2 <= 10
  Repetir
    Escribir "Ingrese Nota 3 (0-10):"
    Leer n3
  Hasta Que n3 >= 0 Y n3 <= 10
  promedio <- (n1 + n2 + n3) / 3
  Si promedio >= 7.0 Entonces
    estado <- "Aprobado"
  Sino
    estado <- "Reprobado"
  FinSi
  Escribir "Guardando en notas.txt: ", nombre, ",", promedio, ",", estado
FinAlgoritmo
```

Diagrama de Flujo



Código en C++:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

void guardarNota(string nom, double n1, double n2, double n3) {
    double prom = (n1+n2+n3)/3.0;
    string est = (prom >= 7.0) ? "Aprobado" : "Reprobado";
    ofstream arch("notas.txt", ios::app);
    arch<<nom<<","<<prom<<","<<est<<"\n";
    arch.close();
    cout<<"Registrado con estado: "<<est<<"\n";
}

int main() {
    guardarNota("Victor", 10, 8, 9);
    return 0;
}
```



Código en Java:

```
import java.io.*;

public class Notas {

    public static void guardar(String nom, double n1, double n2, double n3) {
        double prom = (n1+n2+n3)/3.0;
        String est = (prom >= 7.0) ? "Aprobado" : "Reprobado";
        try(PrintWriter pw = new PrintWriter(new FileWriter("notas.txt",true))){
            pw.println(nom+","+prom+","+est);
            System.out.println("Guardado. Estado: "+est);
        } catch(Exception e){}
    }

    public static void main(String[] args){
        guardar("Victor", 10, 8, 9);
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Aprobado	N1:10, N2:8, N3:9	Prom=9.0 >= 7	Escribe 'Aprobado'
2. Reprobado	N1:5, N2:5, N3:5	Prom=5.0 < 7	Escribe 'Reprobado'
3. Límite	N1:7, N2:7, N3:7	Prom=7.0 >= 7	Escribe 'Aprobado'



EJERCICIO 6: CONTROL DE ASISTENCIA

Enunciado:

Guardar la asistencia de estudiantes. Registrar nombre, fecha y estado. Mostrar historial y contar faltas. *Aplicando la corrección del usuario: Las fechas generadas deben ser de una sola semana laborable continua, sin fines de semana ni repeticiones. *

Análisis del problema:

Para cumplir con la exclusión de fines de semana solicitada, se utilizará un arreglo fijo o lógica de calendario que genere exclusivamente 5 fechas continuas laborables (ej. Lunes a Viernes). Al registrar en asistencia.txt, el ciclo leerá estas fechas para garantizar unicidad y exclusión de sabados/domingos. Luego, al leer el archivo, un condicional sumará +1 al contador de faltas si el estado coincide con 'Falta'.

Variables de entrada, salida y proceso:

Entrada: nombre, estadoAsistencia (Presente/Falta).

Proceso: Iteración sobre array fechas[5], escritura en archivo, lectura con condicional de conteo.

Salida: Archivo persistido, número total de faltas.

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio6_Asiistencia
  Definir fechas Como Cadena
  Dimension fechas[5]
  Definir nombre, estado Como Cadena
  Definir i, faltas Como Entero
  fechas[0] <- "2026-05-25 (Lunes)"
  fechas[1] <- "2026-05-26 (Martes)"
  fechas[2] <- "2026-05-27 (Miercoles)"
  fechas[3] <- "2026-05-28 (Jueves)"
  fechas[4] <- "2026-05-29 (Viernes)"
  Escribir "Ingrese nombre del estudiante:"
  Leer nombre
  Para i <- 0 Hasta 4 Con Paso 1 Hacer
    Repetir
      Escribir "Asistencia el ", fechas[i], " (Presente/Falta):"
      Leer estado
      Hasta Que estado = "Presente" O estado = "Falta"
      Escribir "Escribiendo en asistencia.txt: ", fechas[i], " - ", estado
    FinPara
  Escribir "Calculando faltas desde archivo..."
  faltas <- 0
  Para i <- 0 Hasta 4 Con Paso 1 Hacer
    Escribir "Que estado se leyo del archivo para el dia ", i+1, "?"
    Leer estado
    Si estado = "Falta" Entonces
      faltas <- faltas + 1
```

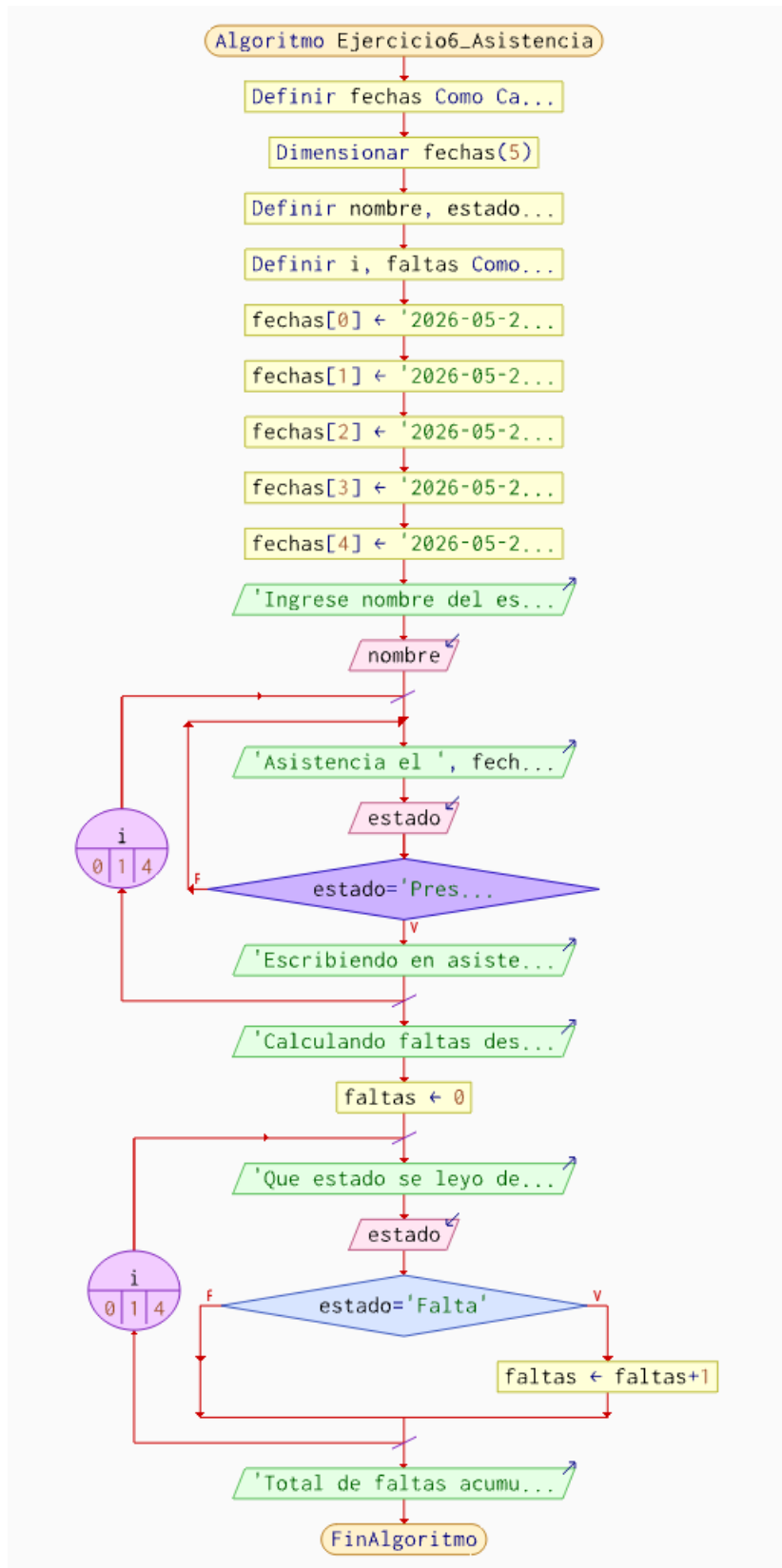


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



FinSi
FinPara
Escribir "Total de faltas acumuladas: ", faltas
FinAlgoritmo

Diagrama de Flujo



Código en C++:

```
#include <iostream>
#include <fstream>
#include <string>
```



```
using namespace std;
void registrarAsistencia(string nombre) {
    // Restriccion: 5 dias laborables, fechas unicas
    string fechas[5] = {"2026-05-25", "2026-05-26", "2026-05-27", "2026-05-28",
"2026-05-29"};
    ofstream arch("asistencia.txt", ios::app);
    for(int i=0; i<5; i++){
        string estado;
        cout<<"Asistencia para "<<fechas[i]<<": "; cin>>estado;
        arch<<nombre<<","<<fechas[i]<<","<<estado<<"\n";
    }
    arch.close();
}

void mostrarFaltas() {
    ifstream arch("asistencia.txt");
    string linea; int faltas = 0;
    while(getline(arch, linea)) {
        if(linea.find("Falta") != string::npos) faltas++;
    }
    cout<<"Total de faltas: "<<faltas<<"\n";
}

int main() { registrarAsistencia("Victor"); mostrarFaltas(); return 0; }
```

Código en Java:

```
import java.io.*;
import java.util.Scanner;
public class Asistencia {
    public static void registrar(String nom) {
        String[] f = {"2026-05-25", "2026-05-26", "2026-05-27", "2026-05-28",
"2026-05-29"};
        Scanner sc = new Scanner(System.in);
        try(PrintWriter pw = new PrintWriter(new FileWriter("asistencia.txt",true))) {
            for(int i=0; i<5; i++){
                System.out.print("Estado para "+f[i]+" : ");
                String est = sc.next();
                pw.println(nom+" "+f[i]+" "+est);
            }
        } catch(Exception e) {}
    }

    public static void main(String[] args){ registrar("Victor"); }
}
```

3 Casos de Prueba de Escritorio:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Validación Fechas	Uso del sistema	Itera arreglo sin Sab/Dom	Registra 5 días exactos.
2. Registro de Faltas	Ingresa 2 Faltas y 3 Presente	El contador detecta 'Falta'	Muestra: Total de faltas: 2
3. Asistencia Perfecta	Ingresa 5 Presentes	Ninguna coincidencia con 'Falta'	Muestra: Total de faltas: 0



EJERCICIO 7: CAJERO AUTOMÁTICO BÁSICO

Enunciado:

Simular un cajero automático utilizando archivos. Guardar usuario y saldo. Consultar saldo, depositar y retirar dinero.

Análisis del problema:

Puesto que es un sistema basado en un archivo de texto secuencial, la 'actualización' del saldo requiere leer el saldo anterior del archivo cajero.txt, sumar o restar el monto en memoria RAM, y luego sobrescribir el archivo (modo ios::trunc o sin el parámetro 'true' en Java) con el nuevo saldo para reflejar el depósito o retiro.

Variables de entrada, salida y proceso:

Entrada: opcion, monto.

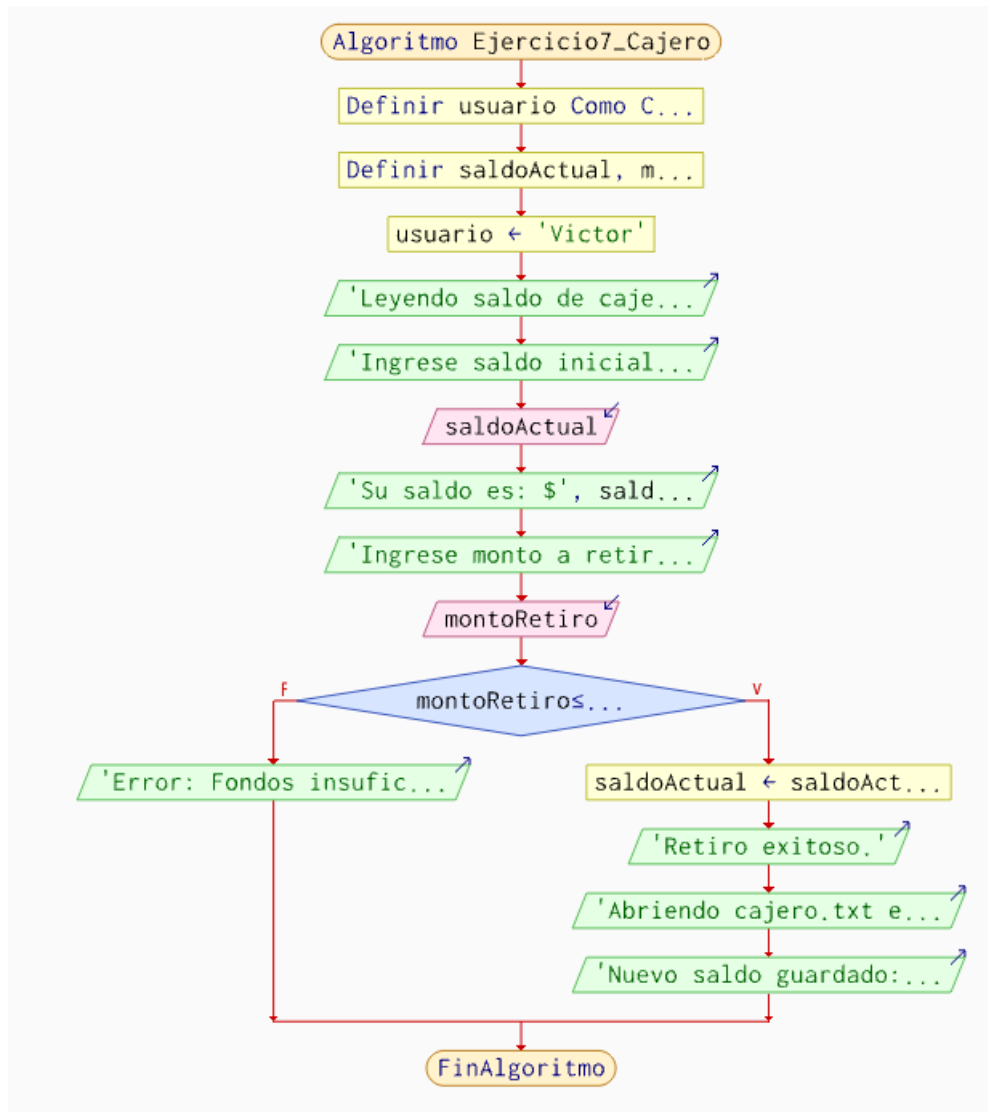
Proceso: leer archivo -> operacion matematica (saldo += deposito) -> escribir archivo modo truncar.

Salida: Archivo actualizado con el saldo final.

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio7_Cajero
  Definir usuario Como Cadena
  Definir saldoActual, montoRetiro Como Real
  usuario <- "Victor"
  Escribir "Leyendo saldo de cajero.txt..."
  Escribir "Ingrese saldo inicial extraido del archivo:"
  Leer saldoActual
  Escribir "Su saldo es: $", saldoActual
  Escribir "Ingrese monto a retirar:"
  Leer montoRetiro
  Si montoRetiro <= saldoActual Entonces
    saldoActual <- saldoActual - montoRetiro
    Escribir "Retiro exitoso."
    Escribir "Abriendo cajero.txt en modo Sobrescribir (Truncate)..."
    Escribir "Nuevo saldo guardado: $", saldoActual
  Sino
    Escribir "Error: Fondos insuficientes. No se modifica archivo."
  FinSi
FinAlgoritmo
```

Diagrama de Flujo



Código en C++:

```
#include <iostream>
#include <fstream>
using namespace std;
double leerSaldo() {
    ifstream arch("cajero.txt");
    string usr; double saldo=0;
    if(arch>>usr>>saldo) return saldo;
    return 0.0;
}
void actualizar(string usr, double saldo) {
    ofstream arch("cajero.txt"); // sobrescribe
    arch<<usr<<" "<<saldo;
}
int main() {
    string usr = "Victor";
```



```
double saldo = leerSaldo(); // simulacion init
if(saldo==0) actualizar(usr, 100.0); // fondo inicial

saldo = leerSaldo();
cout<<"Saldo: "<<saldo<<". Depositando 50...\n";
actualizar(usr, saldo + 50);
return 0;
}
```

Código en Java:

```
import java.io.*;

public class Cajero {
    public static double leer() {
        try(BufferedReader br = new BufferedReader(new FileReader("cajero.txt"))){
            String[] d = br.readLine().split(","); return Double.parseDouble(d[1]);
        }catch(Exception e){ return 0; }
    }

    public static void actualizar(String usr, double s) {
        try(PrintWriter pw = new PrintWriter(new FileWriter("cajero.txt"))){ // sin
            true sobrescribe
            pw.println(usr+","+s);
        }catch(Exception e){}
    }

    public static void main(String[] args){
        actualizar("Victor", 100.0); // init
        double saldo = leer();
        actualizar("Victor", saldo - 20.0); // retiro
        System.out.println("Saldo final: " + leer());
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Depósito	Saldo inicial: 100, Depósito: 50	saldo = 100 + 50	cajero.txt sobrescrito con 150
2. Retiro Valido	Saldo: 150, Retiro: 20	150 - 20 = 130	cajero.txt con 130
3. Retiro Invalido	Saldo: 130, Retiro 500	500 > 130. Condicional rechaza.	No se modifica el txt.



EJERCICIO 8: BIBLIOTECA VIRTUAL

Enunciado:

Registrar libros en un archivo. Guardar código, título, autor y estado. Buscar libros, mostrar disponibles y prestados.

Análisis del problema:

El registro de libros se hace en modo append. Para los reportes específicos (disponibles o prestados), se recorrerá el archivo completo validando si el campo de estado (índice 3 del split) coincide con la palabra 'Disponible' o 'Prestado', omitiendo los que no coincidan.

Variables de entrada, salida y proceso:

Entrada: estadoBuscado.

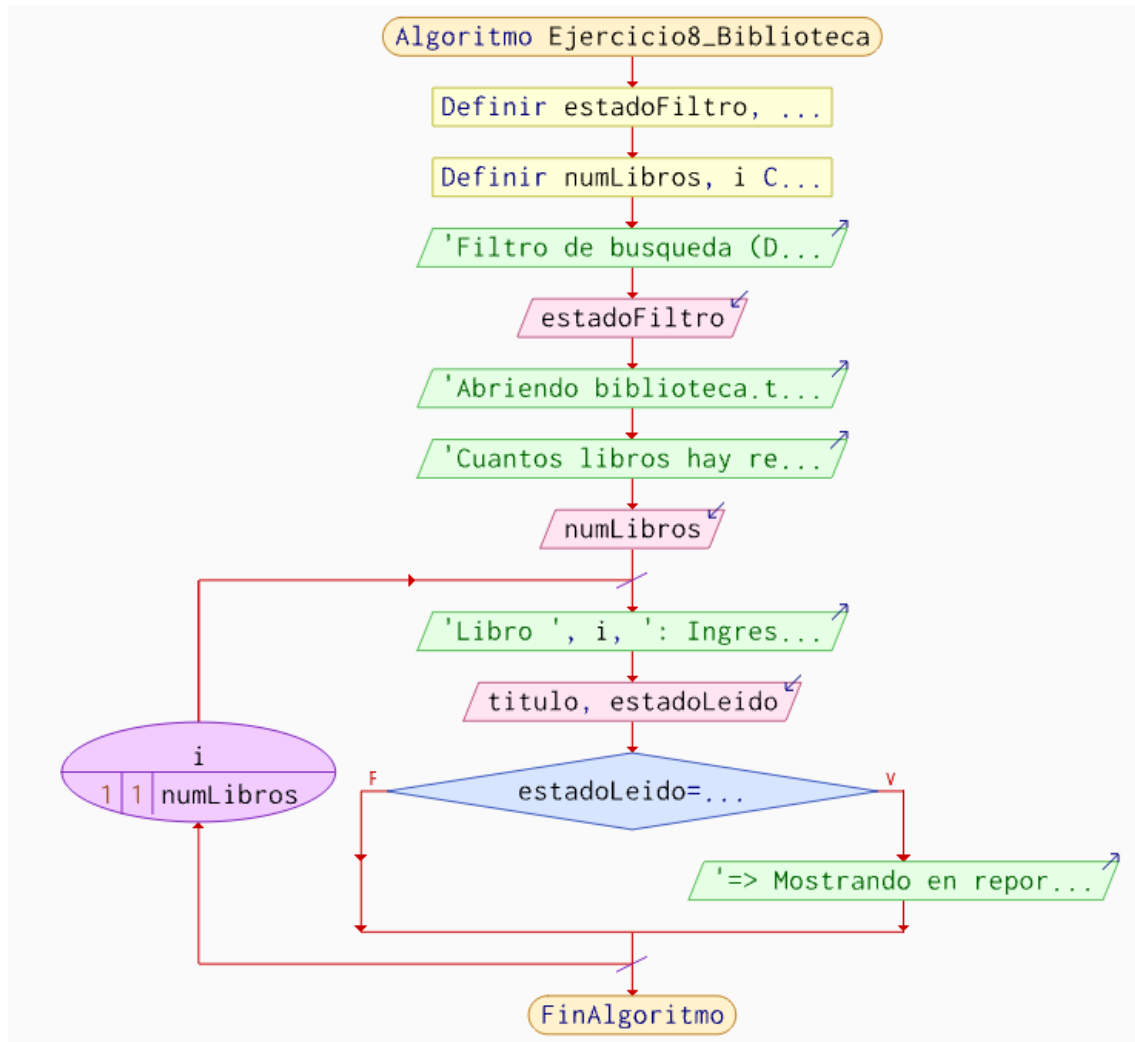
Proceso: Iteración sobre líneas, split(','), comparación de cadena en la posición de estado.

Salida: Listado filtrado de libros.

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio8_Biblioteca
  Definir estadoFiltro, estadoLeido, titulo Como Cadena
  Definir numLibros, i Como Entero
  Escribir "Filtro de busqueda (Disponible/Prestado):"
  Leer estadoFiltro
  Escribir "Abriendo biblioteca.txt..."
  Escribir "Cuantos libros hay registrados?"
  Leer numLibros
  Para i <- 1 Hasta numLibros Con Paso 1 Hacer
    Escribir "Libro ", i, ": Ingrese titulo y estado extraido:"
    Leer titulo, estadoLeido
    Si estadoLeido = estadoFiltro Entonces
      Escribir "=> Mostrando en reporte: ", titulo, " (", estadoLeido, ")"
    FinSi
  FinPara
FinAlgoritmo
```

Diagrama de Flujo



Código en C++:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
void mostrarPorEstado(string estadoBuscado) {
    ifstream arch("biblioteca.txt");
    string linea;
    cout<<"--- Libros "<<estadoBuscado<<" ---\n";
    while(getline(arch, linea)) {
        if(linea.find(estadoBuscado) != string::npos) {
            cout<<linea<<"\n";
        }
    }
}
int main() {
    ofstream arch("biblioteca.txt", ios::app);
    arch<<"L1,Matematicas,AutorX,Disponible\n";
```



```
arch.close();  
mostrarPorEstado("Disponible");  
return 0;  
}
```

Código en Java:

```
import java.io.*;  
public class Biblioteca {  
    public static void mostrarFiltro(String est) {  
        try(BufferedReader br = new BufferedReader(new  
            FileReader("biblioteca.txt"))){  
            String linea;  
            System.out.println("Filtro: " + est);  
            while((linea = br.readLine())!=null){  
                if(linea.contains(est)) System.out.println(linea);  
            }  
        } catch(Exception e){}  
    }  
    public static void main(String[] args){  
        mostrarFiltro("Disponible");  
    }  
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Registrar	L1, Calculo, Disponible	Guarda en txt	Se añade la fila al txt
2. Filtrar Disponible	estado='Disponible'	Solo imprime filas que tengan 'Disponible'	Muestra: L1, Calculo, Disponible
3. Filtrar Prestado	estado='Prestado'	Si no hay prestados, bucle no imprime nada	Lista en blanco



EJERCICIO 9: SISTEMA DE VENTAS

Enunciado:

Crear un sistema de ventas. Registrar producto, cantidad, precio y total. Generar factura y mostrar historial.

Análisis del problema:

La generación de factura implicará dar un formato visualmente amigable al momento de imprimir y guardar en el texto (ej. encabezados y separadores). Se creará la clase Venta que calcula el subtotal y lo envía al flujo de escritura estructurado.

Variables de entrada, salida y proceso:

Entrada: prod, cant, precio.

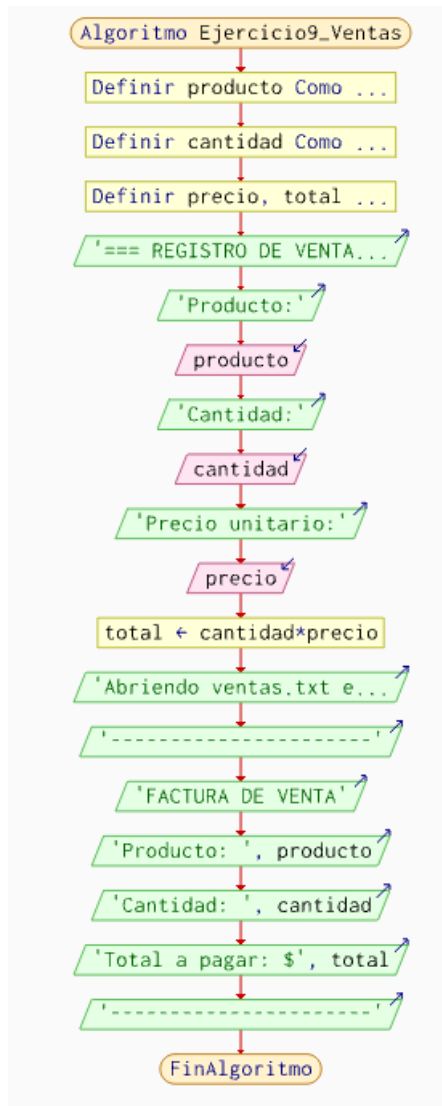
Proceso: total = cant * precio, formateo de texto con saltos de linea.

Salida: Factura impresa en ventas.txt.

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio9_Ventas
  Definir producto Como Cadena
  Definir cantidad Como Entero
  Definir precio, total Como Real
  Escribir "=== REGISTRO DE VENTA ==="
  Escribir "Producto:"
  Leer producto
  Escribir "Cantidad:"
  Leer cantidad
  Escribir "Precio unitario:"
  Leer precio
  total <- cantidad * precio
  Escribir "Abriendo ventas.txt en modo Append..."
  Escribir "-----"
  Escribir "FACTURA DE VENTA"
  Escribir "Producto: ", producto
  Escribir "Cantidad: ", cantidad
  Escribir "Total a pagar: $", total
  Escribir "-----"
FinAlgoritmo
```

Diagrama de Flujo



Código en C++:

```
#include <iostream>
#include <fstream>
using namespace std;

void generarFactura(string prod, int cant, double precio) {
    double total = cant * precio;
    ofstream arch("ventas.txt", ios::app);
    arch<<"===== FACTURA =====\n";
    arch<<"Prod: "<<prod<<" | Cant: "<<cant<<"\n";
    arch<<"Total a Pagar: $"<<total<<"\n";
    arch.close();
    cout<<"Factura generada en ventas.txt\n";
}

int main() {
    generarFactura("Teclado", 2, 25.5);
}
```



```
    return 0;
}
```

Código en Java:

```
import java.io.*;

public class Ventas {
    public static void facturar(String prod, int cant, double precio) {
        double total = cant * precio;
        try(PrintWriter pw = new PrintWriter(new FileWriter("ventas.txt",true))){
            pw.println("--- FACTURA ---");
            pw.println("Prod: "+prod+" | Cant: "+cant+" | Total: $" +total);
        }catch(Exception e){}
    }
    public static void main(String[] args){
        facturar("Monitor", 1, 150.0);
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Venta basica	Teclado, 2, 25.5	$2 * 25.5 = 51$	ventas.txt incluye linea de FACTURA y \$51
2. Multiples ventas	Nueva venta	Append añade nueva factura abajo	2 facturas en txt
3. Historial	Leer txt	Itera el archivo e imprime	Muestra todas las facturas consolidadas



EJERCICIO 10: MINI PROYECTO INTEGRADOR (CRUD)

Enunciado:

Desarrollar un sistema completo aplicando POO y archivos. Implementar CRUD, persistencia, menú interactivo y validaciones.

Análisis del problema:

Un CRUD en archivos de texto requiere un algoritmo especial para 'Actualizar' y 'Eliminar'. Como no se pueden borrar líneas directamente, el algoritmo abre el archivo original (lectura) y un archivo temporal (escritura). Itera línea por línea: si la línea NO es la que el usuario quiere borrar, la copia al temporal. Luego borra el archivo original y renombra el temporal como el original.

Variables de entrada, salida y proceso:

Entrada: idBorrar.

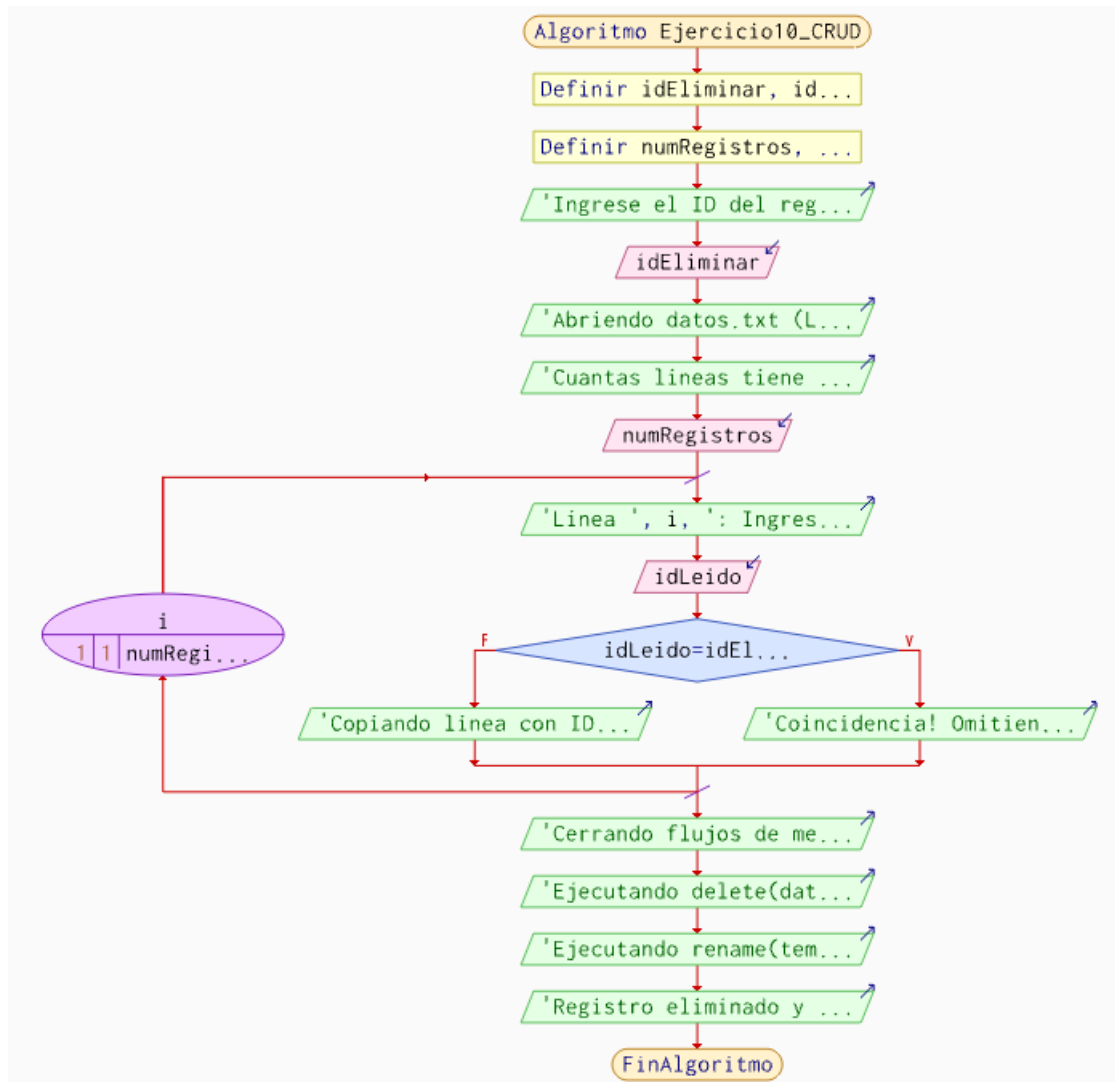
Proceso: copiar lineas validas al temp.txt, delete(original), rename(temp, original).

Salida: Archivo persistente con el registro eliminado o actualizado.

Algoritmo y Pseudocódigo (PSeInt):

```
Algoritmo Ejercicio10_CRUD
  Definir idEliminar, idLeido Como Cadena
  Definir numRegistros, i Como Entero
  Escribir "Ingrese el ID del registro a eliminar:"
  Leer idEliminar
  Escribir "Abriendo datos.txt (Lectura) y temp.txt (Escritura)..."
  Escribir "Cuantas lineas tiene el archivo original?"
  Leer numRegistros
  Para i <- 1 Hasta numRegistros Con Paso 1 Hacer
    Escribir "Linea ", i, ": Ingrese el ID leido:"
    Leer idLeido
    Si idLeido = idEliminar Entonces
      Escribir "Coincidencia! Omitiendo escritura en archivo temporal."
    Sino
      Escribir "Copiando linea con ID ", idLeido, " a temp.txt"
    FinSi
  FinPara
  Escribir "Cerrando flujos de memoria..."
  Escribir "Ejecutando delete(datos.txt)..."
  Escribir "Ejecutando rename(temp.txt -> datos.txt)..."
  Escribir "Registro eliminado y persistido exitosamente."
FinAlgoritmo
```

Diagrama de Flujo



Código en C++:

```
#include <iostream>
#include <fstream>
#include <string>
#include <cstdint>
using namespace std;
void eliminarRegistro(string idBuscar) {
    ifstream arch("datos.txt");
    ofstream temp("temp.txt");
    string linea;
    while(getline(arch, linea)) {
        // Copia todo excepto la linea que queremos eliminar
        if(linea.find(idBuscar) == string::npos) {
            temp<<linea<<"\n";
        }
    }
    arch.close(); temp.close();
}
```



```
remove("datos.txt");
rename("temp.txt", "datos.txt");
cout<<"Registro eliminado correctamente.\n";
}
int main() {
    eliminarRegistro("ID_123");
    return 0;
}
```

Código en Java:

```
import java.io.*;
public class Crud {
    public static void eliminar(String id) {
        File orig = new File("datos.txt");
        File temp = new File("temp.txt");
        try(BufferedReader br = new BufferedReader(new FileReader(orig));
            PrintWriter pw = new PrintWriter(new FileWriter(temp))) {
            String linea;
            while((linea = br.readLine())!=null) {
                if(!linea.contains(id)) pw.println(linea);
            }
        } catch(Exception e){}
        orig.delete();
        temp.renameTo(orig);
    }
    public static void main(String[] args){
        eliminar("ID_123");
    }
}
```

3 Casos de Prueba de Escritorio:

Caso de Prueba	Condición de Entrada	Proceso	Resultado en Salida
1. Eliminar exitoso	id='123', archivo tiene IDs: 123, 456	Copia solo la 456 a temp. Borra y renombra.	datos.txt solo contiene a 456
2. Eliminar Inexistente	id='999'	Copia todas las líneas a temp.	Archivo queda exactamente igual
3. Actualizar	Combina eliminar + crear	Borra línea anterior, inserta línea nueva con mismo ID	Simula el UPDATE en un txt

2.8 Habilidades blandas empleadas en la práctica



- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

El uso de la clase FileWriter configurada con el parámetro append (en true) demostró ser fundamental para añadir nuevos registros a los ficheros sin sobrescribir la información previamente guardada, permitiendo simular el comportamiento de una base de datos persistente.

Delegar las operaciones de lectura y escritura a métodos internos dentro de las clases de Java (en lugar de concentrar todo en el método main) optimizó significativamente la estructura del programa, cumpliendo con las buenas prácticas de la Programación Orientada a Objetos.

El manejo de archivos de texto secuenciales requiere obligatoriamente la implementación de bloques try-catch para capturar excepciones como IOException, lo que comprueba la importancia de gestionar correctamente los errores para evitar que el sistema colapse al no encontrar un archivo.

2.10 Recomendaciones

Utilizar el bloque try-with-resources de Java al instanciar las clases de lectura y escritura; esto garantiza que los flujos de datos se cierren automáticamente al terminar el proceso, evitando bloqueos en el sistema operativo o pérdida de memoria.

Estandarizar el uso de delimitadores (como comas o punto y coma) al momento de guardar los datos en el archivo de texto. Esto facilita enormemente la recuperación de la información usando el método split(",") sin que los espacios en blanco de los nombres generen errores de índice.

Validar la existencia del archivo antes de intentar realizar operaciones de lectura o cálculos matemáticos sobre él. Se recomienda programar el sistema para que informe al usuario con un mensaje amigable si el documento .txt aún no ha sido creado, en lugar de permitir que el programa lance una excepción abrupta.

2.11 Referencias bibliográficas

2.12 Anexos



Link de Github

<https://github.com/VictorChacha1/APE-05-Creacion-Lectura-y-Escritura-de-Ficheros.git>

Evidencias de los ejercicios

Ejercicio1 y Ejercicio2

```
1850024298,Victor,Software,9.0
1802807030,Luis,Software,7.0
1623274038,Maria,Software,10.0
1845689710,Jose,Software,8.0
```

Ejercicio3

```
> ≡ inventario.txt
A1,Mouse,10,15.5
```

Ejercicio4

```
≡ agenda.txt ×
Ejercicio4 > ≡ agenda.txt
1 Victor Chacha,09999999,v@uta.edu.ec
2
```

Ejercicio5

```
≡ notas.txt ×
Ejercicio5 > ≡ notas.txt
1 Victor,9.0,Aprobado
2
```

Ejercicio6



```
≡ asistencia.txt X
Ejercicio6 > ≡ asistencia.txt
1 Victor,2026-05-25,Presente
2 Victor,2026-05-26,Presente
3 Victor,2026-05-27,Falto
4 Victor,2026-05-28,Presente
5 Victor,2026-05-29,Presente
6
```

Ejercicio7

```
≡ cajero.txt X
Ejercicio7 > ≡ cajero.txt
1 Victor,80.0
2
```

Ejercicio8

```
≡ biblioteca.txt X
Ejercicio8 > ≡ biblioteca.txt
1 Cien años de soledad - Disponible
2 Don Quijote de la Mancha - Prestado
```

Ejercicio9

```
≡ ventas.txt X
Ejercicio9 > ≡ ventas.txt
1 FACTURA
2 Prod: Monitor | Cant: 1 | Total: $150.0
3
```

Ejercicio10

```
≡ datos.txt X
Ejercicio10 > ≡ datos.txt
1 ID_123,Victor,10
2 ID_456,Maria,9
```